

“Linux File Descriptor and File I/O Management System”

A Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

**L. SAINATH
T. SHANMUKHA
R. NISHANTH**

**2520090148
2520090050
2520090141**

Under the Esteemed Guidance of

**D. Alekhya
Faculty Guide**

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



Declaration

The Project Report entitled “Linux File Descriptor and File I/O Management System” is a record of bonafide work of L. SAINATH - 2520090148, T. SHANMUKHA - 2520090050, and R. NISHANTH - 2520090141, submitted in partial fulfillment of the requirements for the award of B. Tech in Computer Science and Engineering at K L (Deemed to be) University. The work presented in this report is based on the team’s Linux-based Operating Systems and Systems Programming project.

The implementation and results documented here correspond to the supplied C source program and project problem statement for the 2026–27, Term-I course.

L. SAINATH	2520090148
T. SHANMUKHA	2520090050
R. NISHANTH	2520090141

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini-project report entitled “Linux File Descriptor and File I/O Management System” is a bonafide work done and submitted by L. Sainath - 2520090148, T. Shanmukha - 2520090050, and R. Nishanth - 2520090141,
in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science Engineering, K L (Deemed to be) University, during the academic year 2026–27, Term-I.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

We express our sincere gratitude to everyone who supported the completion of this Operating Systems and Systems Programming project. The project provided a practical opportunity to connect Linux programming concepts with real system behavior through a working C implementation.

We are especially thankful to our faculty guide, D. Alekhya, for the guidance, review, and encouragement provided during the project work. We also thank the Department of Computer Science Engineering for providing the academic environment and facilities required to develop and test the program.

We thank the teaching and non-teaching staff, classmates, friends, and family members who directly or indirectly supported the completion of the project and its documentation.

L. SAINATH 2520090148

T. SHANMUKHA 2520090050

R. NISHANTH 2520090141

Abstract

The Linux File Descriptor and File I/O Management System is a menu-driven C application designed to demonstrate how user programs interact with the Linux kernel through file descriptors, system calls, processes, memory services, inter-process communication, and POSIX synchronization. Its primary file-management module supports creating and opening files, reading and writing data, appending content, changing the current file offset with `lseek()`, displaying active descriptors, closing descriptors, inspecting inode metadata, exploring directories, changing permissions, comparing buffered and unbuffered I/O, querying filesystem space, and using memory-mapped files. The final implementation also integrates course-outcome demonstrations for the OS service layer, process creation and execution, pipes and FIFOs, signals, process memory layout, dynamic allocation, `mmap()`, copy-on-write, threads, race conditions, mutexes, condition variables, semaphores, producer-consumer synchronization, deadlock concepts, and thread-safe file access. The program is implemented for Linux using GCC and POSIX/Linux APIs and provides clear terminal output for each operation. The expected learning outcome is a practical understanding of low-level file management and the relationship between user-space programs, system calls, kernel-managed resources, processes, memory, and concurrent execution.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		4
	Abstract		5
1	Introduction		8–9
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		10–11
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		12
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		13
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		14–15
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	
		5.5 Input and Output Design	
6	Implementation		16–18
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Command-Line User Interface Implementation	
7	Execution and Demonstration		19–25
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	

		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	Conclusion		26
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	References		27
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER - 1

INTRODUCTION

1.1 Background of the Project

Linux exposes files and many kernel-managed resources through integer file descriptors. High-level file libraries are convenient, but they can hide the sequence of low-level operations performed by the operating system. The project therefore uses a C command-line program to expose those operations directly and to show the descriptor returned by `open()`, the movement of the file offset, the transfer of bytes through `read()` and `write()`, and the release of resources through `close()`.

The supplied implementation develops the original file-I/O idea into a broader Operating Systems and Systems Programming demonstrator. Its main menu is organized around six course-outcome modules: OS as a service layer, processes and process control, inter-process communication, memory management, file systems and file I/O, and concurrency and synchronization. This structure makes the project useful both as a file-descriptor manager and as an integrated laboratory-style demonstration of Linux operating-system concepts.

1.2 Problem Statement

High-level file functions hide details such as file-descriptor allocation, access flags, file offsets, permissions, inode metadata, kernel return values, and error handling. Students can use files successfully without seeing how a process requests these services from the kernel. The project addresses this learning gap by implementing file operations through Linux system calls and by displaying the resulting descriptors, data, offsets, metadata, and errors in the terminal.

A second problem addressed by the completed source is that operating-system concepts are often studied separately. The application brings process creation, IPC, virtual memory, file systems, and synchronization into one executable so that their relationships can be demonstrated from a single interface.

1.3 Scope of the Project

- Linux-only command-line execution, with the source explicitly designed for Linux/POSIX APIs.
- Low-level file creation/opening, read, write, append, seek, close, permissions, metadata, directory exploration, filesystem statistics, and memory-mapped file access.
- Process creation and execution using `fork()`, `exec-family` calls, `waitpid()`, process identifiers, and `/proc` process data.
- Inter-process communication through anonymous pipes, named pipes (FIFO), signal handling, process groups, and sessions.
- Memory demonstrations covering process address space, `/proc/PID/maps`, dynamic allocation, `mmap()`, copy-on-write, memory statistics, and page-fault observation.
- POSIX thread demonstrations covering thread creation, race conditions, mutexes, condition variables, semaphores, producer-consumer coordination, deadlock concepts, and mutex-protected file access.
- A terminal menu that reports results and basic errors; no graphical interface, network service, database, or remote file system is included in the supplied implementation.

1.4 Objectives of the Project

1. Understand Linux file descriptors and their role in managing open files and low-level I/O.

2. Implement file creation, opening, reading, writing, appending, seeking, closing, metadata inspection, permissions, and filesystem operations using C and Linux/POSIX APIs.
3. Demonstrate processes, IPC, memory management, file systems, concurrency, and synchronization in one program.
4. Provide a menu-driven interface that makes each operating-system concept easy to execute and observe.
5. Practice return-value checking and user-visible error reporting for kernel and library calls.
6. Compile and execute the complete application using GCC with POSIX thread support.

1.5 Importance of the Application

The value of this application is educational and practical. It turns abstract terms such as descriptor, inode, pipe, copy-on-write, mutex, semaphore, and memory mapping into observable program behavior. Because the demonstrations use actual Linux interfaces, the user can correlate program output with operating-system concepts and inspect how resources are created, shared, synchronized, and released.

1.6 Target Users / Audience

The intended audience is students learning Operating Systems and Systems Programming, particularly those who need a compact Linux laboratory project for practicing system calls and POSIX APIs. It is also suitable for instructors or evaluators who want to demonstrate multiple OS concepts from a single executable without requiring a graphical interface.

CHAPTER - 2

OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

2.1.1 OS as a Service Layer

The CO-1 module shows the operating system as the layer that provides controlled access to process, user, system, and file services. It obtains system information with `uname()`, displays process and user identifiers, creates a file through `open()/write()/close()`, executes shell commands, and reads `/proc/version`. The output explicitly illustrates the path from user space to system calls, kernel space, the file system, and hardware.

2.1.2 Processes and Process Control

The CO-2 module demonstrates `fork()` for process creation, `execvp()` for replacing a child process image, `waitpid()` for parent-child synchronization, selected process information from `/proc/PID/status`, a safe zombie-process demonstration followed by cleanup, and a textual process-lifecycle model.

2.1.3 Inter-Process Communication

The CO-3 module demonstrates one-way byte transfer through an anonymous pipe, file transfer through a pipe, communication through a named FIFO, signal delivery using `sigaction()` and `kill()`, and the relationship between process IDs, process groups, and sessions.

2.1.4 Memory Management

The CO-4 module connects C memory operations with Linux virtual memory. It prints representative code, stack, heap, and static addresses; reads `/proc/PID/maps`; uses `malloc()`, `calloc()`, `realloc()`, and `free()`; creates anonymous mappings with `mmap()`; maps a file with `MAP_SHARED`; demonstrates copy-on-write after `fork()`; reads memory statistics from `/proc/PID/status`; and observes page-fault information. A separate option prints common debugging commands such as `strace`, `valgrind`, and `gdb`.

2.1.5 File Systems and File I/O

The CO-5 module is the core of the project. It manages an application-side descriptor table and supports file creation/opening in multiple modes, reading and writing through a descriptor, append operations, `lseek()`-based offset changes, descriptor listing and closing, `stat()`-based metadata, directory traversal, `chmod()`, buffered stdio, unbuffered `write()`, `statvfs()` filesystem-space information, and the memory-mapped-file demonstration.

2.1.6 Concurrency and Synchronization

The CO-6 module uses POSIX threads and synchronization primitives. It creates and joins threads, demonstrates an unsynchronized shared counter, protects a critical section with a mutex, coordinates producer and consumer roles with a condition variable, restricts concurrent access with a counting semaphore, implements a bounded producer-consumer queue, explains deadlock through circular wait, and serializes shared-file writes with a mutex.

2.2 Existing Methods / Algorithms

The baseline approach described in the project problem statement focuses on individual file operations: open a file, obtain a descriptor, transfer data with `read()/write()`, move the current offset with `lseek()`, and close the descriptor. High-level C functions such as `fopen()/fprintf()/fclose()` provide an alternative buffered interface but do not emphasize the descriptor-level interaction that is central to this project.

The completed source retains the low-level file workflow and adds demonstrations for related kernel mechanisms. Each feature is intentionally small and menu-driven so that one concept can be executed independently, observed, and then compared with other concepts in the same program.

2.3 Proposed Approach

The proposed approach is a single executable with a hierarchical menu. The main menu selects one of six OS concept groups. Each group contains focused demonstrations. User input is collected in the terminal, the corresponding Linux/POSIX API is invoked, return values are checked, and either the requested result or an error message is displayed. Shared utility functions provide consistent headings, input cleanup, pausing, and tracking of file descriptors opened by the application.

2.4 Advantages of the Proposed Approach

- One executable demonstrates the full learning path from user-space requests to kernel-managed resources.
- Direct use of Linux/POSIX APIs makes descriptors, PIDs, mappings, signals, inodes, and synchronization objects visible in program output.
- Modular menus allow demonstrations to be repeated independently without modifying the code.
- File descriptors opened by the application are registered in a small table and are closed during normal program exit.
- Potentially disruptive concepts are demonstrated safely: the zombie is collected with `waitpid()`, and the deadlock option explains circular wait instead of deliberately hanging the application.
- The code can be compiled with a single GCC command and does not depend on an external GUI framework or database.

CHAPTER - 3

SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Requirement for this Project
Processor	Any system capable of running a modern Linux environment and GCC.
Memory	Sufficient RAM for Linux, compilation, and small process/thread demonstrations; the project itself uses modest memory.
Storage	A small amount of writable disk space for the executable and demonstration files created by the program.
Display/Input	Terminal or console with keyboard input for the menu-driven interface.

3.2 Software Requirements

Software	Purpose
Linux / Ubuntu	Target operating-system environment for Linux system calls, /proc, FIFOs, POSIX processes, and filesystem APIs.
GCC	Compiles the C source code into the executable.
POSIX Threads	Required by the concurrency module; linked through the GCC -pthread option.
Terminal / Bash	Runs the executable and allows inspection of files and process information.
Optional debugging tools	The source prints examples for strace, valgrind, and gdb as learning/debugging aids.
Git / GitHub	Listed in the project form as a version-control/repository tool; no repository URL was supplied in the provided material.

3.3 Operating System Environment – Ubuntu / WSL

The source file explicitly states “Linux only.” It relies on Linux/POSIX facilities such as /proc, fork(), pipe(), mkfifo(), mmap(), statvfs(), pthreads, and Linux process information. Ubuntu is therefore a direct target environment. A Windows Subsystem for Linux setup can be used only when it provides the required Linux APIs and filesystem behavior; the project source itself is written against the Linux environment rather than native Windows APIs.

3.4 Development Tools and Compilers

Compilation and execution commands used by the source code

```
gcc -Wall -Wextra -pthread linux_os_manager.c -o linux_os_manager
./linux_os_manager
```

The source was also compiled during preparation of this documentation using the command above, and the resulting executable ran successfully in the Linux environment used for verification.

CHAPTER - 4

TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

The implementation is written in C and uses standard C library functions together with Linux and POSIX APIs. C is used for direct descriptor handling, process control, pointer-based memory operations, explicit buffer management, thread functions, and synchronization primitives.

4.2 GCC Compiler

GCC is the compiler specified by the source. The options `-Wall` and `-Wextra` enable additional compiler diagnostics, while `-pthread` enables the compilation and linking support required by POSIX threads. The build produces a single executable named `linux_os_manager`.

4.3 Linux System Calls

Area	Representative APIs Used in the Source
System / process information	<code>uname()</code> , <code>getpid()</code> , <code>getppid()</code> , <code>getuid()</code> , <code>getgid()</code> , <code>getpgrp()</code> , <code>getsid()</code>
File I/O	<code>open()</code> , <code>read()</code> , <code>write()</code> , <code>lseek()</code> , <code>close()</code> , <code>stat()</code> , <code>chmod()</code> , <code>ftruncate()</code> , <code>statvfs()</code>
Processes	<code>fork()</code> , <code>execlp()</code> , <code>waitpid()</code>
IPC / signals	<code>pipe()</code> , <code>mkfifo()</code> , <code>sigaction()</code> , <code>kill()</code>
Memory	<code>mmap()</code> , <code>munmap()</code> , <code>msync()</code> , <code>malloc()</code> , <code>calloc()</code> , <code>realloc()</code> , <code>free()</code>
Directories	<code>opendir()</code> , <code>readdir()</code> , <code>closedir()</code>

4.4 POSIX Threads / Synchronization

The concurrency module uses `pthread_create()` and `pthread_join()` for thread lifecycle management, `pthread_mutex_lock()/unlock()` for mutual exclusion, `pthread_cond_wait()/signal()` for condition-based coordination, and `sem_init()/sem_wait()/sem_post()/sem_destroy()` for counting-semaphore control. The program also maintains mutexes and condition variables as global synchronization objects and destroys the relevant objects before normal exit.

4.5 Linux Commands and Utilities

- `ls`, `pwd`, `whoami`, `date`, and `uname -a` are shown as examples in the shell-command demonstration.
- `cat /proc/<PID>/maps` and `cat /proc/<PID>/status` are printed as ways to inspect process memory and status.
- `df -T <path>` is suggested by the filesystem-information module to inspect filesystem type.
- `strace`, `valgrind`, and `gdb` are printed by the memory-debugging option as optional diagnostic tools.

4.6 Other Tools / Libraries Used

The source includes headers for file control, process management, memory mapping, filesystem statistics, directory access, signals, pthreads, semaphores, time, and path limits. The project does not require an external GUI or third-party runtime library beyond the system and POSIX facilities available in the Linux development environment.

CHAPTER - 5

SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The architecture is layered. The user interacts with a menu-driven C application in user space. The selected feature invokes standard C, POSIX, or Linux interfaces. System calls cross into the Linux kernel, where process, memory, IPC, filesystem, and scheduling services manage the underlying resources. Results are returned to the application and displayed in the terminal.

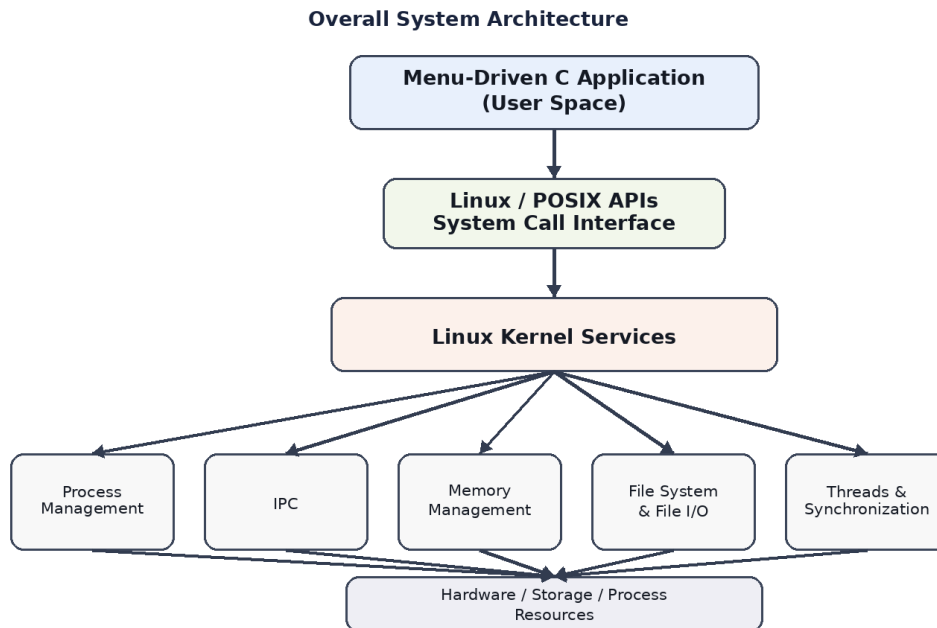


Figure 5.1: Overall system architecture of the Linux OS management application

5.2 System Workflow

At startup, the program initializes the application-side file-descriptor table. The main menu remains active until the user chooses Exit. A module and feature are selected, input is read, the corresponding OS operation is executed, and the result or error is displayed. Control then returns to the menu. On exit, descriptors still registered by the application are closed and synchronization objects are destroyed.

System Workflow

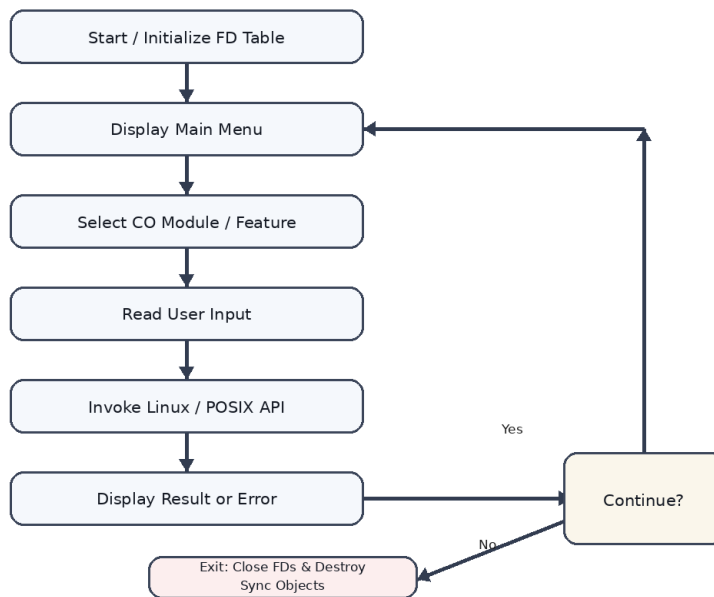


Figure 5.2: High-level workflow from user selection to Linux/POSIX operation

5.3 Flowchart / Data Flow

For the core file-I/O path, data flows as: user input → filename/access mode → open() → file descriptor → read()/write()/lseek() → Linux virtual filesystem and filesystem implementation → file data/metadata → return value or errno → terminal output. For process and IPC features, the flow additionally includes child processes, pipes/FIFOs, and signals. For concurrency features, multiple pthreads share process memory and are coordinated by mutexes, condition variables, or semaphores.

5.4 Module Description

Module	Functions Demonstrated
CO-1: OS Service Layer	System information, system-call demonstration, shell command execution, /proc system interface.
CO-2: Process Management	fork(), exec(), process information, zombie cleanup, process lifecycle.
CO-3: IPC	Anonymous pipe, file transfer via pipe, FIFO, signal handling, process groups/sessions.
CO-4: Memory Management	Address space, memory maps, dynamic allocation, anonymous mmap, mapped file, copy-on-write, memory statistics, page faults, debugging commands.
CO-5: File System & I/O	Create/open, read, write, append, seek, descriptor table, close, metadata/inode, directory explorer, permissions, buffered/unbuffered I/O, filesystem information, mapped file.
CO-6: Concurrency	Threads, race condition, mutex, condition variable, semaphore, producer-consumer, deadlock concept, thread-safe file access.

5.5 Input and Output Design

Input is accepted through scanf() for numeric menu choices and descriptors and through fgets() for filenames, commands, paths, and data. A clear_input() utility consumes leftover newline characters so that mixed numeric and line input works across menus. Output uses consistent section headers and descriptive labels. Errors from many system calls are reported with perror(), allowing the user to see failures such as invalid descriptors, inaccessible paths, or permission problems.

CHAPTER - 6

IMPLEMENTATION

6.1 Module-wise Implementation

6.1.1 Global Descriptor Tracking and Utilities

The program defines `MAX_OPEN_FILES` as 20 and keeps two parallel arrays: `fd_table[]` for descriptors and `fd_names[][]` for the corresponding paths. `initialize_fd_table()` marks all slots as unused, `register_fd()` stores a newly opened descriptor, and `unregister_fd()` clears a slot after `close()`. Utility functions also handle input cleanup, screen pauses, horizontal separators, and formatted headers.

Descriptor-table data structures used by the implementation

```
#define MAX_OPEN_FILES 20
#define BUFFER_SIZE 4096

int fd_table[MAX_OPEN_FILES];
char fd_names[MAX_OPEN_FILES][PATH_MAX];

void initialize_fd_table(void) {
    for (int i = 0; i < MAX_OPEN_FILES; i++) {
        fd_table[i] = -1;
        fd_names[i][0] = '\0';
    }
}
```

6.1.2 Core File Descriptor Operations

`create_open_file()` obtains a filename and one of four access modes: read-only, write-only/create, read-write/create, or write/create/append. It then calls `open()` with mode 0644 where creation is possible. Successful descriptors are registered. `read_file_demo()` and `write_file_demo()` accept an existing descriptor and transfer bytes. `seek_demo()` reports the current offset and moves it relative to the beginning, current position, or end. `close_file_demo()` closes and unregisters the descriptor.

Representative low-level file operations

```
int fd = open(filename, flags, 0644);
if (fd == -1) {
    perror("open");
    return;
}
register_fd(fd, filename);

ssize_t bytes = write(fd, data, strlen(data));
off_t current = lseek(fd, 0, SEEK_CUR);
```

6.1.3 Metadata, Directories, Permissions, and Filesystem Information

`stat()` is used to print file size, inode number, link count, ownership, permissions, block size, and block count. Directory entries are read with `opendir()/readdir()/closedir()`. `chmod()` updates permissions from an octal value and `stat()` is called again to display the new permission string. `statvfs()` supplies block size and capacity information for a path. The module also contrasts buffered `fprintf()/fclose()` with unbuffered `open()/write()/close()`.

6.2 Algorithm / Pseudocode

6.2.1 Main Control Algorithm

```
INITIALIZE file descriptor table
```

```

DISPLAY project title and implemented COs
REPEAT
    DISPLAY main menu
    READ choice
    IF choice is 1..6 THEN
        CALL corresponding CO submenu
    ELSE IF choice is complete demonstration THEN
        DISPLAY integrated OS flow
    ELSE IF choice is process information THEN
        DISPLAY current process details
    ELSE IF choice is Exit THEN
        CLOSE any registered descriptors
        DESTROY synchronization objects
        TERMINATE
UNTIL program terminates

```

6.2.2 File I/O Algorithm

```

READ filename and access mode
fd = open(filename, flags, permissions)
IF fd < 0: display error and return
REGISTER fd
READ/WRITE bytes as requested
OPTIONALLY move current offset using lseek()
OPTIONALLY inspect metadata using stat()
close(fd)
UNREGISTER fd

```

6.2.3 Producer-Consumer Algorithm

```

PRODUCER repeats for values 1..10:
    lock queue mutex
    while queue is full: wait on not_full
    insert item; signal not_empty
    unlock queue mutex

CONSUMER repeats 10 times:
    lock queue mutex
    while queue is empty: wait on not_empty
    remove item; signal not_full
    unlock queue mutex

```

6.3 Core C / C++ Implementation

The complete supplied source contains the menus, utility functions, all six CO modules, and the main() loop in one C translation unit. The program uses a 4096-byte general buffer for several file transfers, PATH_MAX-sized arrays for paths, and a fixed descriptor table of 20 application-tracked entries. Each demonstration is implemented as a separate function to keep the menu logic readable and allow isolated testing.

6.4 Linux System Calls / APIs Used

API	Role in Implementation
open / read / write / lseek / close	Primary low-level descriptor and byte I/O operations.
stat / chmod / statvfs	File metadata, permission modification, and filesystem-capacity information.
fork / execlp / waitpid	Process creation, program execution, and child collection.
pipe / mkfifo	Anonymous and named IPC channels.
sigaction / kill	Signal-handler installation and signal delivery.

mmap / munmap / msync / ftruncate	Anonymous and file-backed virtual-memory mappings.
opendir / readdir / closedir	Directory traversal.
pthread_* / sem_*	Thread creation, joining, mutual exclusion, condition signaling, and semaphore control.

6.5 Process / Thread Implementation

Process demonstrations create a child with `fork()`. In the process-creation example the child reports its PID and parent PID, sleeps briefly, and exits; the parent waits with `waitpid()`. In the `exec` example the child replaces itself with “ls -l” through `execlp()`. The thread demonstrations create `pthread_t` objects, pass small integer IDs to worker functions, and use `pthread_join()` so the main demonstration waits for every thread to finish.

6.6 IPC / Synchronization Implementation

The anonymous-pipe demonstration closes unused pipe ends in each process, writes a message in the parent, and reads it in the child. A second pipe routine transfers file contents. The FIFO routine creates `/tmp/linux_os_manager_fifo`, forks, opens the FIFO in complementary modes, transfers a message, waits for the child, and unlinks the FIFO. Signal handling installs a `SIGUSR1` handler with `sigaction()` and sends the signal to the current process with `kill()`.

Synchronization is implemented with a global mutex, a global condition variable, a counting semaphore initialized to three permits, and a separate mutex/condition-variable pair for the producer-consumer queue. The race-condition option deliberately increments a shared counter without synchronization to show why protection is needed; the mutex option serializes updates and produces a final counter of 10 in its ten-thread demonstration.

6.7 Memory Management Implementation

Dynamic allocation is shown using `malloc()`, `realloc()`, `calloc()`, and `free()`. The anonymous `mmap()` option requests a 4096-byte read/write private mapping, writes a string, prints its address and size, and releases it with `munmap()`. The file-mapping option creates/truncates a file, maps it with `MAP_SHARED`, copies data into the mapping, synchronizes it with `msync()`, unmaps it, and closes the descriptor. Additional functions read `/proc` process files to display virtual-memory mappings and selected memory statistics.

6.8 Command-Line User Interface Implementation

The user interface is entirely terminal-based. Every CO has a submenu with numbered options and a Back entry. The main menu also provides a “Complete Project Demonstration” that prints a recommended learning sequence from opening a file and obtaining a descriptor through processes, pipes, `/proc`, memory mapping, inode information, threads, mutexes, semaphores, and producer-consumer synchronization.

CHAPTER - 7

EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

Save the supplied source as `linux_os_manager.c`, open a terminal in the same directory, and compile with GCC. The source header provides the following command:

```
gcc -Wall -Wextra -pthread linux_os_manager.c -o linux_os_manager
```

For this documentation, the supplied source was compiled successfully with this command in a Linux environment. The `-pthread` option is necessary because the CO-6 module uses POSIX threads and synchronization primitives.

7.2 Execution Commands

```
./linux_os_manager
```

The executable prints the project banner, lists the six implemented COs, and displays the main menu. A user can enter a menu number, execute one or more demonstrations, return to the main menu, and choose 0 to exit.

```
Program Main Menu

#####
#
#   LINUX FILE DESCRIPTOR AND FILE I/O MANAGEMENT SYSTEM   #
#
#   OPERATING SYSTEM PROJECT                               #
#
#####

Implemented COs:
CO-1 : OS as a Service Layer
CO-2 : Processes and Process Control
CO-3 : Inter-Process Communication
CO-4 : Memory Management
CO-5 : File Systems and File I/O
CO-6 : Concurrency and Synchronization

=====
MAIN MENU
=====
1. CO-1 - OS Service Layer
2. CO-2 - Process Management
3. CO-3 - IPC
4. CO-4 - Memory Management
5. CO-5 - File Systems & File I/O
6. CO-6 - Concurrency & Synchronization
7. Complete Project Demonstration
8. Display Current Process Information
0. Exit
```

Figure 7.1: Main menu displayed by the compiled program

7.3 Sample Inputs and Outputs

7.3.1 File Descriptor and File I/O Example

A representative run opened `report_demo.txt` in read/write mode and returned file descriptor 3. The program wrote 39 bytes, used `lseek()` to move the offset from 39 back to 0, read the same 39 bytes, displayed the descriptor in the application table, and printed file metadata including a 39-byte size and `-rw-r--r--` permissions. Descriptor values, inode values, user IDs, and block counts are environment-dependent, so those numeric fields can differ on another machine.

```
File Descriptor and File I/O Demonstration

C0-5: CREATE / OPEN FILE
=====
Enter filename:
1. Read Only
2. Write Only
3. Read + Write
4. Append
Select mode:
File opened successfully.
Filename      : report_demo.txt
File Descriptor: 3

C0-5: WRITE FILE
=====
Enter file descriptor: Enter data to write:
39 bytes written successfully.

Press ENTER to continue...
=====
C0-5: FILE SYSTEMS AND FILE I/O

C0-5: FILE OFFSET USING lseek()
=====
Enter file descriptor: Current offset: 39

1. Beginning
2. Current + N
3. End
Select: New file offset: 0

Press ENTER to continue...
=====
C0-5: FILE SYSTEMS AND FILE I/O

C0-5: READ FILE
=====
Enter file descriptor:
Bytes read: 39
Data:
-----
Hello from the Linux File I/O project.

Press ENTER to continue...
=====
C0-5: FILE SYSTEMS AND FILE I/O

C0-5: FILE DESCRIPTOR TABLE
=====
FD      File
-----
3       report_demo.txt

Standard file descriptors:
0 -> stdin
1 -> stdout
2 -> stderr

Press ENTER to continue...

C0-5: FILE METADATA / INODE
=====
Enter filename: Filename      : report_demo.txt
Size           : 39 bytes
```

Figure 7.2: Verified file open, write, seek, read, descriptor table, and metadata output

7.3.2 Process and IPC Example

The process-creation option demonstrates separate parent and child outputs around `fork()` and then collects the child with `waitpid()`. The anonymous-pipe option creates a pipe before forking, closes the unused ends, sends a message from the parent, receives it in the child, and reports completion.

Process Creation and Anonymous Pipe

```
C0-2: PROCESS CREATION USING fork()
=====
Parent PID before fork : 743

[CHILD]
Child PID : 744
Parent PID : 743
Child process terminating...

#####
#                                     #
#   LINUX FILE DESCRIPTOR AND FILE I/O MANAGEMENT SYSTEM   #
#                                     #
#   OPERATING SYSTEM PROJECT                               #
#                                     #
#####

Implemented C0s:
C0-1 : OS as a Service Layer
C0-2 : Processes and Process Control
C0-3 : Inter-Process Communication
C0-4 : Memory Management

C0-3: ANONYMOUS PIPE
=====

[CHILD]
Received through pipe: Hello from Parent through Anonymous Pipe!

#####
#                                     #
#   LINUX FILE DESCRIPTOR AND FILE I/O MANAGEMENT SYSTEM   #
#                                     #
#   OPERATING SYSTEM PROJECT                               #
#                                     #
#####

Implemented C0s:
C0-1 : OS as a Service Layer
C0-2 : Processes and Process Control
C0-3 : Inter-Process Communication
C0-4 : Memory Management
C0-5 : File Systems and File I/O
C0-6 : Concurrency and Synchronization

=====
MAIN MENU
=====
```

Figure 7.3: Process creation and anonymous-pipe demonstration output

7.3.3 Memory Example

The dynamic-memory option allocates five integers, stores and prints values, expands the allocation to ten integers with `realloc()`, creates zero-initialized memory with `calloc()`, and finally releases the allocated blocks. The printed addresses naturally vary between executions.

```
Dynamic Memory Allocation Demonstration

CO-4: DYNAMIC MEMORY ALLOCATION
=====
malloc() allocated 5 integers.
Address: 0x56325746c2c0
Values: 1 2 3 4 5

realloc() expanded memory to 10 integers.
New address: 0x56325746c2c0
calloc() allocated zero-initialized memory.
Address: 0x56325746c2f0

free() released allocated memory.

Press ENTER to continue...
=====
CO-4: MEMORY MANAGEMENT
=====
1. Process Address Space
2. /proc/PID/maps Memory Map
3. Dynamic Memory Allocation
4. Anonymous mmap()
5. Memory-Mapped File
6. Copy-on-Write
7. Memory Statistics
```

Figure 7.4: Dynamic memory allocation demonstration

7.4 Screenshots of Execution

The following verified output shows representative concurrency behavior. Thread ordering can vary because scheduling is concurrent; therefore the exact order of “running,” “completed,” and semaphore-acquisition messages may differ between runs even when the program is correct.

POSIX Threads and Synchronization

C0-6: POSIX THREAD CREATION

```
=====
Thread 1 running.
Thread 2 running.
Thread 3 running.
Thread 2 completed.
Thread 1 completed.
Thread 3 completed.
```

All threads completed.

Press ENTER to continue...

C0-6: CONCURRENCY AND SYNCHRONIZATION

C0-6: MUTEX SYNCHRONIZATION

```
=====
Thread 1 acquired mutex.
Thread 1 updated shared counter.
Thread 2 acquired mutex.
Thread 2 updated shared counter.
Thread 3 acquired mutex.
Thread 3 updated shared counter.
Thread 4 acquired mutex.
Thread 4 updated shared counter.
Thread 5 acquired mutex.
Thread 5 updated shared counter.
Thread 7 acquired mutex.
Thread 7 updated shared counter.
Thread 8 acquired mutex.
Thread 8 updated shared counter.
Thread 6 acquired mutex.
Thread 6 updated shared counter.
Thread 9 acquired mutex.
Thread 9 updated shared counter.
Thread 10 acquired mutex.
Thread 10 updated shared counter.
```

Final counter: 10

Press ENTER to continue...

C0-6: COUNTING SEMAPHORE

```
=====
Thread 1 waiting for semaphore...
Thread 1 acquired resource.
Thread 2 waiting for semaphore...
Thread 2 acquired resource.
Thread 3 waiting for semaphore...
Thread 3 acquired resource.
Thread 4 waiting for semaphore...
Thread 5 waiting for semaphore...
Thread 6 waiting for semaphore...
Thread 1 releasing resource.
Thread 2 releasing resource.
Thread 3 releasing resource.
Thread 4 acquired resource.
Thread 5 acquired resource.
Thread 6 acquired resource.
Thread 4 releasing resource.
Thread 5 releasing resource.
Thread 6 releasing resource.
```

Only 3 threads could access the resource simultaneously.

Press ENTER to continue...

C0-6: PRODUCER-CONSUMER

```
=====
Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Consumer consumed: 2
Producer produced: 3
Consumer consumed: 3
Producer produced: 4
Producer produced: 5
Consumer consumed: 4
Producer produced: 6
Consumer consumed: 5
Producer produced: 7
Producer produced: 8
Consumer consumed: 6
Producer produced: 9
```

Figure 7.5: POSIX threads, mutex, semaphore, and producer-consumer output

The integrated demonstration option summarizes how the project connects system calls, process management, IPC, memory management, filesystems, and scheduling/threads.

```

Complete OS Concept Demonstration

COMPLETE OS CONCEPT DEMONSTRATION
=====

C0-1 -> System Call Layer
C0-2 -> Process Creation
C0-3 -> Inter-Process Communication
C0-4 -> Memory Management
C0-5 -> File System and File I/O
C0-6 -> Threads and Synchronization

Suggested demonstration flow:

1. Open a file
2. Obtain file descriptor
3. Write data using write()
4. Change offset using lseek()
5. Read using read()
6. Create child using fork()
7. Communicate using pipe()
8. Display /proc process information
9. Display process memory map
10. Demonstrate mmap()
11. Display inode information
12. Create POSIX threads
13. Protect shared data with mutex
14. Demonstrate semaphore
15. Demonstrate producer-consumer

This represents the complete OS interaction journey:

USER PROGRAM
|
v
SYSTEM CALLS
|
v
LINUX KERNEL
|
+----- PROCESS MANAGEMENT
|
+----- MEMORY MANAGEMENT
|
+----- IPC
|
+----- FILE SYSTEM
|
+----- SCHEDULING / THREADS
|
v
HARDWARE / STORAGE

Press ENTER to continue...
=====
MAIN MENU
=====
1. C0-1 - OS Service Layer
2. C0-2 - Process Management

```

Figure 7.6: Complete OS concept demonstration printed by the application

7.5 Demonstration of OS Concepts

Concept	Observable Evidence in the Program
System-call boundary	The CO-1 system-call demonstration creates a file with open()/write()/close() and prints a user-space → kernel-space model.
Process control	Parent/child PIDs, fork(), exec(), waitpid(), zombie cleanup, and /proc process status are displayed.
IPC	Pipe, file transfer through a pipe, FIFO, and SIGUSR1 signal handling are executed from the menu.
Memory management	Process addresses, /proc maps, allocation, mmap, copy-on-write, memory statistics, and page-fault information are shown.

File system	Descriptors, offsets, bytes, metadata/inodes, directories, permissions, buffered/unbuffered I/O, and filesystem capacity are displayed.
Concurrency	Threads, race condition, mutex-protected counter, condition variable, semaphore, producer-consumer queue, deadlock model, and thread-safe file writes are demonstrated.

CHAPTER - 8

CONCLUSION

8.1 Summary of the Project

The project delivers a working Linux command-line application centered on file descriptors and low-level file I/O, while also integrating the major OS concepts represented by the six course-outcome modules in the supplied source. The implementation makes kernel-facing operations visible through descriptive terminal output and groups the demonstrations into a repeatable menu structure.

8.2 Objectives Achieved

- Implemented and demonstrated Linux file descriptors with `open()`, `read()`, `write()`, `lseek()`, and `close()`.
- Implemented file metadata, permissions, directory traversal, buffered/unbuffered I/O, filesystem statistics, and memory-mapped file access.
- Demonstrated process creation/execution, IPC, signals, memory management, threads, and synchronization mechanisms.
- Provided a single menu-driven application for practical OS concept demonstration.
- Handled many API failures with explicit return-value checks and `perror()` messages.
- Confirmed that the supplied program builds with GCC using `-Wall -Wextra -pthread` and executes in Linux.

8.3 Results Achieved

The supplied source compiled successfully during report preparation. Representative execution confirmed the main menu, low-level file open/write/seek/read workflow, application descriptor table, inode metadata, fork-based process creation, anonymous-pipe communication, dynamic allocation, POSIX thread creation, mutex-protected updates, counting-semaphore behavior, and producer-consumer synchronization. The program also contains additional menu options documented in earlier chapters.

8.4 Skills and Learning Outcomes

The project provides hands-on practice with Linux system-call programming, descriptor lifecycle management, process and IPC APIs, virtual memory, file-system metadata, POSIX threads, synchronization, terminal I/O, compiler diagnostics, and structured C program design. It also demonstrates the importance of resource cleanup, return-value checking, and synchronization when multiple execution contexts share data or files.

CHAPTER - 9

REFERENCES

9.1 Books

No book citation was supplied in the uploaded project materials. The implementation and this report are grounded primarily in the provided project problem statement and the supplied C source code.

9.2 Research Papers

No research paper was supplied or required as an implementation dependency for this project. The work is a systems-programming demonstration based on Linux/POSIX interfaces.

9.3 Linux / GNU Documentation

- Linux manual pages for the APIs used in the source, including `open(2)`, `read(2)`, `write(2)`, `lseek(2)`, `close(2)`, `fork(2)`, `wait(2)`, `pipe(2)`, `mmap(2)`, `stat(2)`, `chmod(2)`, and related calls.
- POSIX/Linux manual documentation for `pthread_create()`, `pthread_join()`, mutexes, condition variables, and semaphores.
- GCC compiler documentation / local gcc help for the compilation flags used by the source.

9.4 Online Resources

No external website or repository URL was supplied with the project files. The project form lists Git/GitHub as a planned version-control tool, but the GitHub repository field is blank. If a repository is created later, its URL can be added here and to the project submission form.

Primary project artifacts used for this report: (1) “OS project code (1).txt” containing the complete C implementation, and (2) “OSSP_Abstract_Filled (2).docx” containing the project title, team details, initial abstract, objectives, methodology, tools, and expected outcome.